Exercise: Run Cars CRUD Tests on Pull Requests

Goal: Write a GitHub Actions workflow that automatically runs the Cars CRUD application's existing **unit, integration, API, and end-to-end (E2E)** tests whenever a pull request targets `main`.

Prerequisites: You have completed GITHUB_ACTIONS_GUIDE.md and TESTING_WORKSHOP.md. The tests from the workshop should already be committed to `main` (as in the earlier GitHub exercise).

1. Prepare a branch

From the root of `cars-crud/`:

```
git switch main
git pull --ff-only origin main
git switch -c github-actions-tests
```

If `github-actions-tests` already exists, switch to it instead of creating it again.

2. Requirements

Create `.github/workflows/tests.yml`. It must:

1. Run for **pull requests targeting** `main`, including when new commits are pushed to that PR. Do not add a `push` trigger to this exercise.
2. Use an **Ubuntu** runner and **Python 3.12**.
3. Download the repository files onto the runner.
4. Install the application's test dependencies from `requirements-dev.txt`.
5. Install **Playwright Chromium and its Linux system dependencies**.
6. Execute all four test groups, preferably as separate named steps:
    - `tests/unit/`
    - `tests/integration/`
    - `tests/api/`
    - `tests/e2e/`
7. Show a **failed** workflow run if any test fails, and a **successful** run when all tests pass.

You do **not** need a separate SQLite service, to launch Uvicorn yourself, or to configure authentication. The provided fixtures create isolated databases, and the E2E fixture starts its own FastAPI server.

3. Starter outline

Fill in the missing parts using the guide and hints below. **This is intentionally incomplete**, not a finished workflow:

```
name: Cars CRUD Tests

# TODO: add the pull_request event filtered to the main branch
```

```yaml
permissions:
  contents: read

jobs:
  tests:
    runs-on: ubuntu-latest
    steps:
      # TODO: checkout the repository
      # TODO: set up Python 3.12
      # TODO: install requirements-dev.txt
      # TODO: install Chromium and its Linux dependencies
      # TODO: run unit tests
      # TODO: run integration tests
      # TODO: run API tests
      # TODO: run E2E tests
```

4. Hints

Hint A: Trigger and branch. Use the `pull_request` event and its `branches` filter. Here, `main` is the **PR target**, not the source branch `github-actions-tests`.

Hint B: Prepare the runner. Reuse the `uses:` steps from the beginner guide:

- `actions/checkout@v7` to access the code.
- `actions/setup-python@v7` with `python-version: "3.12"`.

Hint C: Install Python dependencies. The project has **`requirements-dev.txt`**, which also includes the normal application requirements:

```
python -m pip install -r requirements-dev.txt
```

Hint D: Prepare the browser. This project uses Chromium, not Firefox or WebKit. On the Ubuntu runner, install both Chromium and the necessary OS libraries:

```
python -m playwright install --with-deps chromium
```

Hint E: Run the four groups. These are the same commands you used in the testing workshop:

```
python -m pytest tests/unit -q
python -m pytest tests/integration -q
python -m pytest tests/api -q
python -m pytest tests/e2e -q
```

Translate each needed shell command into a YAML step using `- name:` and `run:`. By default, a command returning a non-zero exit status fails its step, so do not suppress test errors or add `continue-on-error`.

5. Push and open a pull request

After completing the YAML file:

```
git add .github/workflows/tests.yml
git commit -m "Run Cars CRUD tests on pull requests"
git push -u origin github-actions-tests
```

On GitHub:

1. Select **Compare & pull request** or **Pull requests > New pull request**.
2. Set **base:** `main` and **compare:** `github-actions-tests`.
3. Create the pull request.
4. Open its **Checks** section (or the **Actions** tab), select the **Cars CRUD Tests** run, and inspect each step's output.
5. Confirm unit, integration, API, and E2E tests all pass.

The workflow should also run again when you push further commits to the open pull request.

6. Prove that a failing test is detected

1. In **`tests/unit/test_schemas.py`**, find `test_color_is_optional`.
2. Temporarily change `assert car.color == ""` to `assert car.color == "Blue"`.
3. Commit and push the change to `github-actions-tests`.
4. Open the PR checks and find the **failed** unit-test step. Later steps may be skipped because the job stopped at the failure.
5. Restore `assert car.color == ""`, commit, and push again.
6. Verify that the **newest** run is successful, then merge the PR into `main`.

Do not leave the intentionally failing assertion in the merged code.

7. Completion checklist

- [ ] My workflow is stored at `.github/workflows/tests.yml`.
- [ ] It triggers only for pull requests targeting `main`.
- [ ] It configures Ubuntu and Python 3.12.
- [ ] It installs `requirements-dev.txt` and Playwright Chromium.
- [ ] All four test groups pass in the GitHub run.
- [ ] Deliberately breaking one unit test produces a failed check.
- [ ] Fixing the test produces a successful new check.
- [ ] The completed pull request is merged into `main`.

Troubleshooting

Problem	Hint
No run appears on the PR	Confirm the file path, valid YAML, enabled Actions, and that the PR targets `main`.
`No module named pytest`	Install `requirements-dev.txt` before running tests.
Chromium executable is missing	Run the Playwright installation step before the E2E tests.
Linux shared-library error	Use `python -m playwright install --with-deps chromium`, not only `install chromium`.
Browser cannot reach the app	Check the E2E fixture output. It starts the local server automatically; no separate application-start step is needed.
A later group did not run	An earlier step may have failed; inspect that step's logs first.

Reference: Playwright Python: running in GitHub Actions.